

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/339442578>

Box-Counting Method in Python for Fractal Analysis of Biomedical Images

Conference Paper · February 2020

DOI: 10.1109/IT48810.2020.9070454

CITATIONS

14

READS

3,509

3 authors:



Ivana Konatar

University of Donja Gorica

3 PUBLICATIONS 15 CITATIONS

SEE PROFILE



Tomo Popovic

University of Donja Gorica

175 PUBLICATIONS 1,389 CITATIONS

SEE PROFILE



Natasa Popovic

University of Montenegro

75 PUBLICATIONS 602 CITATIONS

SEE PROFILE

Box-Counting Method in Python for Fractal Analysis of Biomedical Images

Ivana Konatar, Tomo Popovic, *Senior Member, IEEE*, and Natasa Popovic

Abstract – This paper presents the implementation of a Python-based library with a purpose to determine fractal dimension of biomedical images. The described method is based on the assumption that the images are already pre-processed and contain binarized version of fractal-like structures that can often be found in biomedical images. Three variants of the box-counting method were implemented using different ways for selecting and sampling the boxes: standard non-overlapping box scanning, gliding or overlapping box scanning, and random box sampling. The utility of the proposed software was validated through the analysis of an open access library of binarized images of retinal microvasculature and by the comparison of these results with those obtained by using ImageJ program, that is commonly used for this purpose.

Keywords – applied computer science, box-counting dimension, fractal analysis, fractal dimension, image processing, Python

I. INTRODUCTION

In 1970s, Benoît B. Mandelbrot introduced a formal approach to the analysis of fractal phenomena and pointed out the fact that fractal geometry is present all around us [1,2]. Fractal geometries can be found in particle trajectories, hydrodynamic lines of flux, waves, landscapes, mountains, islands and rivers, rocks, metals, and composite materials, plants, polymers, gels, etc [2]. In past decades, the understanding of fractals and analysis of their properties found applications in various areas of science [3,4], among others, in medicine, for the analysis of biomedical images [5-7]. One such application is assessing the complexity of retinal microvascular morphologies as potential biomarkers of hypertension and diabetes, both by analysis of the image of a whole retina, or of the specific regions of interest [8-11].

The aim of the presented work was to create a Python library for assessing properties of fractal-like structures captured in digital images. Another motivation was to learn Python and provide a useful tool for fractal analysis of biomedical images that can be easily combined with other tools already available in Python-based scientific computing ecosystem. The scope of the research presented in this paper encompasses estimation of fractal dimension of 2D binarized

images of fractal-like structures and validation of this estimation by using well-known fractals and real-life images of retinal microvasculature obtained from public databases. Three variants of box-counting method for fractal dimension calculation were implemented and the validation results are provided and discussed.

II. MATERIALS AND METHODS

Fractal analysis is a mathematical method used to calculate the fractal dimension, a measure of complexity of fractal-like geometry found in nature [2]. It characterizes the self-similarity over different scales, which means that the fractal-like structures display similar patterns at different scales or magnifications [3]. Mathematically, this property is described with equation (1):

$$N(\varepsilon) = \text{const} \cdot \varepsilon^{-D} \quad (1)$$

where $N(\varepsilon)$ represents a measure of the fractal structure pattern at scale ε ; D represents the fractal dimension that tells us how many new similar fractal patterns can be observed as the scale increases or decreases.

In the box-counting method, the object of interest is covered with rectangular box-like grid with boxes with side length ε [12]. The process is repeated for several different scales ε , and for each ε , according to (1), we count and record $N(\varepsilon)$ that corresponds to the number of boxes needed to completely cover the fractal-like structure at the given scale ε . Fractal dimension D , can be derived from (1) and mathematically expressed as:

$$D = \lim_{\varepsilon \rightarrow 0} \frac{\log N(\varepsilon)}{-\log \varepsilon} \quad (2)$$

When using digital images, different scales ε are being used, expressed as the number of pixels, and for each ε , $N(\varepsilon)$ is the number of boxes needed to completely cover the fractal. Fractal dimension D , i.e. box-counting dimension, is determined as the slope of the log-log relationship between measure $N(\varepsilon)$ at scales ε , so that the power law given in (1) is satisfied. In the presented paper, the following three variants of the box-counting method were implemented: 1) the standard box-counting method, 2) the gliding box-counting method and 3) the random placement box-counting method. All three described methods are based on the assumption that the images are already pre-processed and contain binarized version of fractal-like structures.

A. Standard Box-Counting Method

The algorithm for box-counting estimation of fractal

*Research supported in part by the Ministry of Science of Montenegro, scientific project DEMONSTRATE, Grant Agreement No. 01-781/2.

I. Konatar is with Faculty of Applied Sciences, University of Donja Gorica, Oktoih 1, 81000 Podgorica, Montenegro (e-mail: ivana.konatar@udg.edu.me).

T. Popovic is with Faculty for Information Systems and Technologies, University of Donja Gorica, Oktoih 1, 81000 Podgorica, Montenegro (e-mail: tomo.popovic@udg.edu.me).

N. Popovic is with Faculty of Medicine, University of Montenegro, Krusevac bb, 81000 Podgorica, Montenegro (e-mail: npopovic@ucg.ac.me).

dimension for 2D fractal objects is implemented using the following main steps:

Step 1. Let E represent the set of all scales, that is, the set of all square sizes by which we will cover the given image. The maximum value for $\epsilon \in E$ may be $\min(n, m)$, where n and m are the length and width of the image, while the smallest value for ϵ is limited by the image resolution, i.e. the smallest possible value must be greater than or equal to 1px. We start from the largest (or smallest) scale, value of ϵ , and we reduce (or increase) it according to a predetermined algorithm in order to form the set E .

Step 2. Let $N(\epsilon)$ represent the total number of squares, whose sides are equal to ϵ , required to cover the fractal object we are characterizing. Starting from some initial value for ϵ , we cover the whole image with squares that touch. For each square, it is determined whether the fractal structure intersects with it. If that is the case, the value of $N(\epsilon)$ will be increased by 1, which is essentially the reason for the name box-counting. This is illustrated in Figure 1 below.

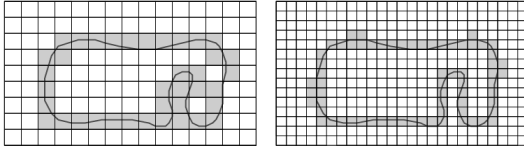


Figure 1. Box-counting method illustration (Steps 2 and 3).

Step 3. Then the same image is overlaid with identical squares of smaller (or larger) sizes ϵ . And for each ϵ , the procedure is repeated as in step 2. When we obtained all the values of $N(\epsilon)$, for each ϵ , we organized these intermediate results as shown in Table I.

TABLE I
SCALE SIZE VS. BOX COUNTS

ϵ	$N(\epsilon)$	ϵ	$N(\epsilon)$
2	228128	32	1024
4	65536	64	256
8	16384	128	64
16	4096	256	16

Step 4. The number of filled squares $N(\epsilon)$ is shown on the log-log graph, as a function of the magnitude of the data from Table I. The log-log chart displays points $\log \epsilon$ vs. $\log N(\epsilon)$, as illustrated in Figure 2.

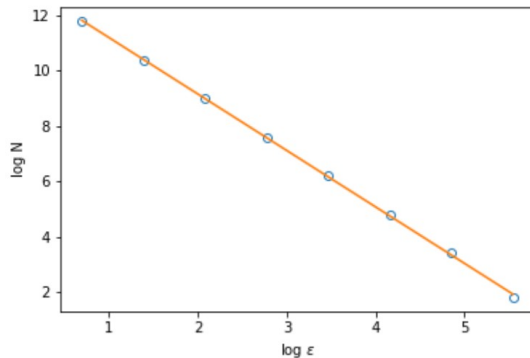


Figure 2. Box-counting method illustration (Steps 4 and 5).

Step 5 As for the last step, we connect the points with the fitted line obtained using the least-squares method, and the slope of that line gives us the estimate for of the fractal dimension D of our graphical object. We take the negative value of the slope coefficient of this line (also in Figure 2).

Higher slope of the fitted line (fractal dimension) indicates that the object has more “fractal” structure, ie. that it becomes more complex as the size of the box decreases, while a lower slope value indicates that the object is more similar to the simple straight line (less “fractal”) and that the amount of detail does not increase very much if we increase the scale.

B. Gliding Box-Counting Method

In this approach, the boxes “slide” over each other. More precisely, we do not fit the new box at the ends of the previous box, but the new box will partially overlap with the previous box. The box scrolling is performed according to a predetermined algorithm, with the same scroll step, both vertically and horizontally when placing the box, until we cover the whole image. The remaining steps to determine the fractal dimension are the same in the standard box-counting method. This approach is illustrated in Figure 3.

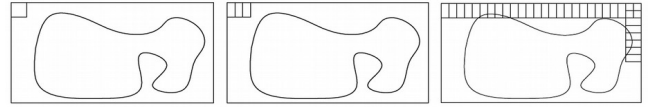


Figure 3. The “gliding” box-counting approach.

C. Random Placement Box-Counting Method

This approach implies that the position of the boxes is chosen at random. In order to stay within the boundaries of a given image, we need to be careful when choosing which coordinates can be selected as the initial position of the box. It is important to note that in the random approach we have to use a large number of boxes than the standard box-counting method, so that there is enough boxes to cover the fractal object we are analyzing even though the boxes overlap. This number of boxes depends on image size and box-size ϵ .

The remaining steps to determine the fractal dimension are the same as in the previous two approaches. This approach is illustrated in Figure 4.

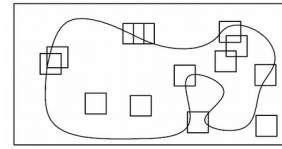


Figure 4. The random box-counting approach.

D. Biomedical Images of Retinal Microvasculature

The STARE database is publicly available [13] and contains 402 raw color retinal images captured with a TopCon TRV-50 fundus camera at 35 degrees field of view. This database contains a subset of 20 raw images that were manually segmented by two experts, Adam Hoover (AH segmented set) and Valentina Kouzentsova (VK segmented set) [14]. Therefore, each raw color image in this subset is

associated with two manually segmented binarized images of the vascular network, one provided by AH and the other by VK. Ten of these raw images represent healthy retinas, and 10 represent retinas with pathological manifestations.

In addition, we used a third set of manually segmented images made available by N. Popovic et al. [15,16]. This set contains 37 images (NP segmented set): standard set of 20 images corresponding to AH i VK sets, and additional 17 images that correspond to the 2 specific retinal pathologies- hypertensive retinopathy (HR) and proliferative diabetic retinopathy (PDR).

E. The Use of ImageJ and FracLac

ImageJ equipped with FracLac plugin is a public domain program written in Java and it is commonly used tool for fractal analysis of biomedical images [17,18]. The three sets of manually segmented images and their fractal dimensions were determined by using ImageJ as reported previously [16]. This information was used to validate the fractal analysis results generated by our Python script.

F. Development Tools, Python and Jupyter

In order to develop the present code, we used the Python programming language and the Jupyter Notebook scientific computing environment that allows collaborative experimenting with the code while all the intermediate results can be accessed and interacted with using a standard web browser [19,20]. Statistical post-processing was done in LibreOffice Calc spreadsheet program.

III. RESULTS AND DISCUSSION

A. Initial Testing, “Sanity Check”

A “sanity check” was performed by the analysis of fractal images with a well known fractal dimension as shown in Table II. The table shows results obtained from all three methods versus the expected theoretical value (Hausdorff dimension) for these fractal images. All three methods provided results that were very close to the expected values.

B. Tuning of the Random Box-Counting Method

As expected, the random box-counting method showed some inconsistency and repeatability issues. It produces different results for different attempts, especially for fractals structures that do not fill much space. Thus, the larger the difference in the number of black and white pixels, such as in the case of Koch curve, the more unreliable the random box-counting method seemed to be. Initially, for this method, we used the number of random boxes equal to the smallest number of squares of a given side ε , which would cover the whole image. That is, we used the number $M=Ps/Pk$, where Ps is the area of the image we analyze and Pk is the area of the square with size ε . By increasing the number of randomly placed boxes 10, 20, and 50 times, we dramatically improved the reliability and repeatability of the random box-counting method. This is illustrated in Table III that shows results of an experiment with an image of Koch fractal.

TABLE II
FRACTAL DIMENSION CALCULATED FOR WELL KNOWN FRACTALS

Fractal	BC ^a	GL ^b	Rand ^c	HD ^d	Error
Koch curve	1.26	1.22	1.25	1.26	±0.04
Julia set z^2-1	1.28	1.30	1.28	1.27	±0.03
Apollonian gasket	1.30	1.35	1.39	1.31	±0.08
Viscek fractal	1.44	1.46	1.45	1.46	±0.02
Sierpinski triangle	1.55	1.57	1.59	1.58	±0.03
Cantor set	1.66	1.68	1.70	1.63	±0.07
Pascal triangle	1.60	1.60	1.60	1.63	±0.03
Asymmetric Cantor flake	1.64	1.69	1.71	1.69	±0.05
Hexaflake	1.76	1.77	1.75	1.77	±0.02
Pentaflake	1.83	1.84	1.82	1.86	±0.04
Sierpinski carpet	1.88	1.89	1.91	1.89	±0.02

a. BC - Box-Counting Method
b. GL - liding Box-Counting Method,
c. Rand - Random Box-Counting Method
d. HD - Housdorff Dimension (theoretical value)

TABLE III
TUNING OF THE RANDOM BOX-COUNTING METHOD

Number of boxes ^a	Attempt ^b					Error ^c
	1	2	3	4	5	
1 x M^ε	1.33	1.29	1.20	1.26	1.21	±0.07
10 x M	1.27	1.24	1.25	1.27	1.26	±0.02
20 x M	1.27	1.26	1.25	1.26	1.26	±0.01
50 x M	1.26	1.26	1.26	1.26	1.26	0

a. M is number of boxes, sized ε by ε pixels, needed to cover the whole image
b. All attempts performed on the same digital image of Koch fractal
c. The theoretical value of Koch curve fractal dimension is 1.26

C. Comparison with ImageJ, Biomedical Images

The implementation of our Python library was validated by comparing its results with ImageJ program while analyzing biomedical images containing retinal microvasculature.

First, we used the built-in Fractal Box Counter, invoked by the “Analyze→Tools→Fractal Box Count...” menu option. When using the same setting values for scale ε , ImageJ yielded near identical results as the Python script.

Next, we used the FracLac plugin with the default settings to analyze AH, VK and NP sets, where each set contained 20 manually segmented images of retina [14,16]. The default settings for scale size ε go to up to 45% of the image size [18]. Then, we repeated the analysis with the box size settings tuned to the settings of our Python script: we used the scale size ε going from 2 pixels to up to 20% of the image size. These results are summarized in Table IV.

Finally, we evaluated our library by repeating the experiment from [16], where an extended set of manually segmented images of retina was used. These pictures came from three groups of patients: those with normal retina (N), those with hypertensive retinopathy (HR), and the patients with proliferative diabetic retinopathy (PDR). The results of the fractal analysis are shown in Table V.

TABLE IV
COMPARISON WITH IMAGEJ SOFTWARE, RETINAL MICROVASCULATURE

Dataset	Tool	Normal ^d	Pathology ^d
AH set [14] (20 images)	ImageJ ^a	1.59±0.02	1.50±0.06
	ImageJ ^b	1.49±0.03	1.41±0.06
	BC ^c	1.48±0.02	1.41±0.04
	GL ^c	1.48±0.02	1.41±0.04
	Rand ^c	1.48±0.02	1.41±0.05
VK set [14] (20 images)	ImageJ ^a	1.70±0.02	1.60±0.06
	ImageJ ^b	1.64±0.03	1.51±0.09
	BC ^c	1.60±0.02	1.48±0.07
	GL ^c	1.60±0.02	1.48±0.07
	Rand ^c	1.60±0.02	1.49±0.08
NP set [16] (20 images)	ImageJ ^a	1.61±0.02	1.52±0.06
	ImageJ ^b	1.51±0.03	1.41±0.07
	BC ^c	1.48±0.02	1.40±0.06
	GL ^c	1.48±0.02	1.40±0.06
	Rand ^c	1.48±0.02	1.40±0.07

a. ImageJ with the default settings
b. ImageJ with scales settings matched to our Python library
c. Results obtained using different methods from our Python library
d. Results show mean fractal dimension ± standard deviation

TABLE V
COMPARISON WITH IMAGEJ SOFTWARE, DIABETIC RETINOPATHY

Dataset	Tool	N ^d	HR ^d	PDR ^d
NP extended set [16]: Normal, HR, PDR (27 images)	ImageJ ^a	1.61±0.02	1.54±0.06	1.55±0.05
	ImageJ ^b	1.51±0.03	1.43±0.06	1.46±0.07
	BC ^c	1.48±0.02	1.42±0.05	1.43±0.06
	GL ^c	1.48±0.02	1.42±0.05	1.43±0.06
	Rand ^c	1.47±0.02	1.41±0.06	1.43±0.06

a. ImageJ with the default settings
b. ImageJ with scales settings matched to our Python library
c. Results obtained using different methods from our Python library
d. Results show mean fractal dimension ± standard deviation

IV. CONCLUSION

The paper describes a research effort to implement a Python library for determining fractal dimension of 2D binarized fractal objects. The initial version of the library provides three variants for the box-counting method that estimate fractal dimension. Initial tests showed that the results produced by all three methods are very close to the expected theoretical values. The library was validated by analyzing images of retinal microvasculature from public databases. These calculations were compared to the results obtained by ImageJ program. After the settings were matched, we obtained a very good alignment of the results and showed that this new tool can be used for similar experiments with biomedical images.

The library needs additional refining with respect to the usage interface, such as function argument options, and refactoring to clear and optimize code. Further research will

include adding options to perform calculations on various pre-selected regions of interest, batch processing, statistical post-processing of results, and visualization of results.

REFERENCES

- [1] B. B. Mandelbrot, "The Fractal Geometry of Nature," W.H. Freeman and Co., New York, 1982.
- [2] J-F. Gouyet, "Physics and Fractal Structures," Springer, 1996.
- [3] K. Falconer, "Fractal Geometry: Mathematical Foundations and Applications", John Wiley & Sons, New York, 1990.
- [4] M. F. Barnsley, "Fractals Everywhere", Dover Publications, Inc., Mineola, New York, 2012.
- [5] R. W. Glenny, H. T. Robertson, S. Yamashiro, and J. B. Bassingthwaite, "Fractal analysis in medicine," Journal of Applied Physiology, 70(6), pp 2351-2367, 1991.
- [6] N. Rajković, B. Krstonošić, and N. Milošević, "Box-Counting Method of 2D Neuronal Image: Method Modification and Quantitative Analysis Demonstrated on Images from the Monkey and Human Brain," Computational and Mathematical Methods in Medicine, Hindawi, 2017:8967902, pp 1-9, 2017.
- [7] C. C. Chen, J. S. Daponte, M. D. Fox, "Fractal feature analysis and classification in medical imaging," IEEE Transactions on Medical Imaging, IEEE, 8(2), pp133-142, 1989.
- [8] C. Cheung et al., "Quantitative and qualitative retinal microvascular characteristics and blood pressure," Journal of Hypertension, 29(7), pp 1380–1391, 2011.
- [9] S. Țălu, D. Călugăru, and C. Lupașcu, "Characterisation of human non-proliferative diabetic retinopathy using the fractal analysis," International Journal of Ophthalmology 8(4) , pp 770–776, 2015.
- [10] T. J. Macgillivray, N. Patton F. N. Doubal, C. Graham, and J. M. Wardlaw, "Fractal analysis of the retinal vascular network in fundus images," Conf. Proc. IEEE. Eng. Med. Biol. Soc. Pp 6456–6459, 2007.
- [11] N. Popovic, S. Vujosevic, and T. Popovic, "Regional Patterns in Retinal Microvascular Network Geometry in Health and Disease," Scientific Reports, 9(2019):16340, Nature Research, 2019.
- [12] L. S. Liebovitch, T. Toth, "A fast algorithm to determine fractal dimensions by box counting," Phys. Lett. A 141, 386–390, 1989.
- [13] The STARE Project. Accessed January 2020. [Online]. Available: <http://cecas.clemson.edu/~ahoover/stare/>
- [14] A. Hoover, V. Kouznetsova, M. Goldbaum, "Locating blood vessels in retinal images by piecewise threshold probing of amatched filter response," IEEE Transactions on Medical Imaging 19 (2000), IEEE, pp 203–210, 2000.
- [15] N. Popovic, M. Radunovic, J. Badnjar, T. Popovic, "Manually segmented vascular networks from images of retina with proliferative diabetic and hypertensive retinopathy," Data in Brief 18(2018), Elsevier, pp 470-473, 2018.
- [16] N. Popovic, M. Radunovic, J. Badnjar, T. Popovic, "Fractal dimension and lacunarity analysis of retinal microvascular morphology in hypertension and diabetes," Microvasc. Res. 118 (2018), Elsevier, pp 36–43, 2018.
- [17] C. Schneider, W. Rasband, K. Eliceiri, "NIH image to ImageJ: 25 years of image analysis," Nat. Methods 9 (2012), pp 671–675, 2012.
- [18] A. Karperien, "FracLac for ImageJ-FracLac Advanced User's Manual," National Institutes of Health. Accessed January 2020. [Online]. Available: <http://rsb.info.nih.gov/ij/plugins/fraclac/fraclac-manual.pdf>
- [19] Python programming language. Accessed January 2020. [Online]. Available: <https://www.python.org>
- [20] Project Jupyter. Accessed January 2020. [Online]. Available: <http://jupyter.org>